Procesos de Desarrollo de Software

Integración continua

Sesión Práctica

Contenidos

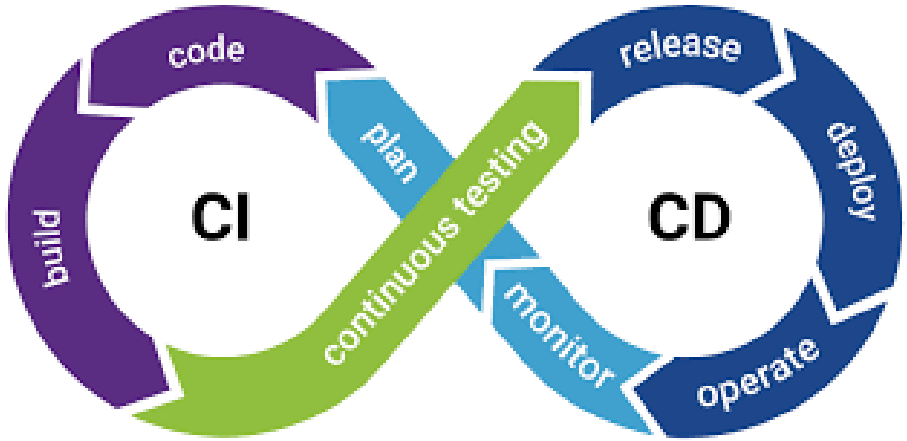
1. Integración continua con GitHub Actions
2. Configuración de SonarQube.
3. ArchUnit

PARTE I
Integración continua

con GitHub Actions

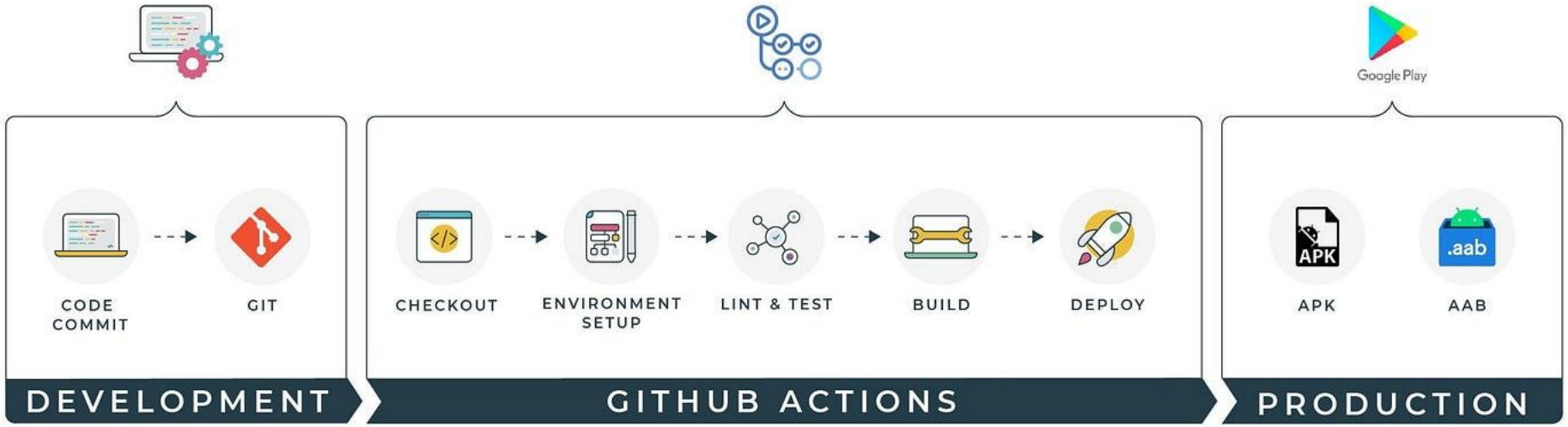
Moticación

- ¿Qué es la integración continua?
  - Es el procedimiento por el cual se integra código en el repositorio con frecuencia y de manera mayormente automatizada
  - Cada incremento de código debe contar con automatismos que comprueben su corrección
- Existen diversas plataformas de integración continua:
  - Gitlab CI/CD
  - Github actions
  - Jenkins
  - Azure DevOps
  - AWS CodePipeline
  - etc



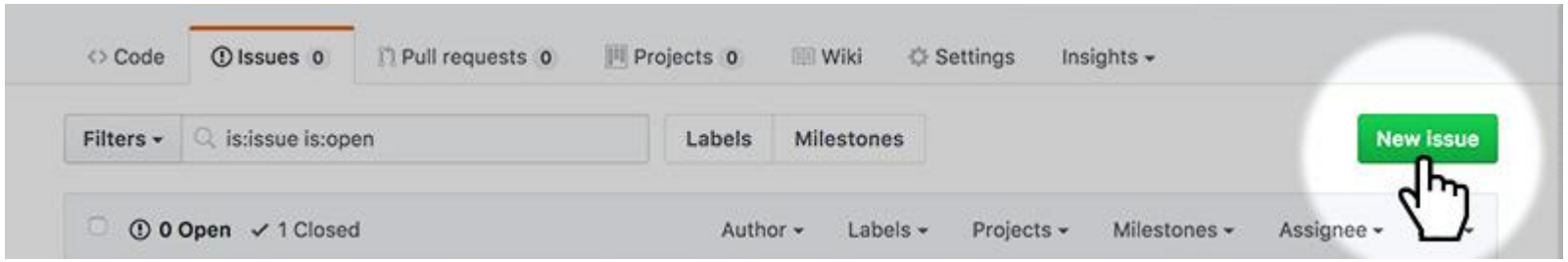
Proceso de desarrollo típico (en GitHub)

- Un equipo de trabajo **debe estar organizado** con un sistema para gestionar el trabajo y realizar los incrementos de código
- Github ofrece:
  - **Issues**: Mecanismo de tickets para organizar el trabajo a realizar
  - **Projects**: Planificar el desarrollo completo de un proyecto o funcionalidad
  - **Pull request**: Sistema para solicitar incluir un incremento de código en una rama
  - **Github Actions**: Acciones a realizar según eventos del sistema CI/CD



Proceso de desarrollo típico (en GitHub)

- Veamos un ejemplo
  - Se quiere implementar una pequeña mejora en nuestro software
  - Para ello se crea una nueva ISSUE con el detalle del trabajo a realizar



  - Un miembro del equipo decide ocuparse de ella, para ello:
    - Actualiza el repositorio local del proyecto a lo último que hay en el repositorio
    - Crea una nueva rama a partir de la actual en cuyo nombre incluye el código de la issue

```
git checkout -b pds-issue-5
```

Proceso de desarrollo típico (en GitHub)

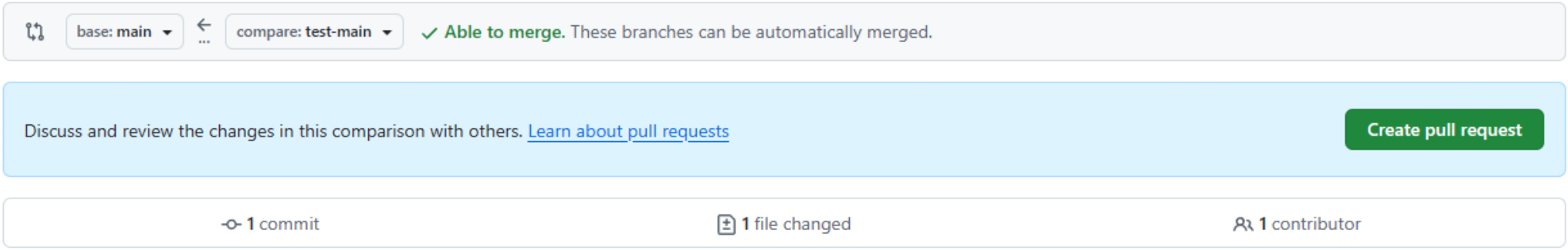
- Veamos un ejemplo
  - El desarrollador trabaja en la rama haciendo commits con cada cambio relevante

```
git commit -a -m “Refactorizar CSS”
```

  - Adicionalmente sube su rama al repositorio github para evitar que sólo esté en su equipo, sincronizando los commits que lleva realizados en local

```
git push origin pds-issue-5
```

  - Cuando su trabajo está terminado, en github crea una solicitud de integración de código Pull Request

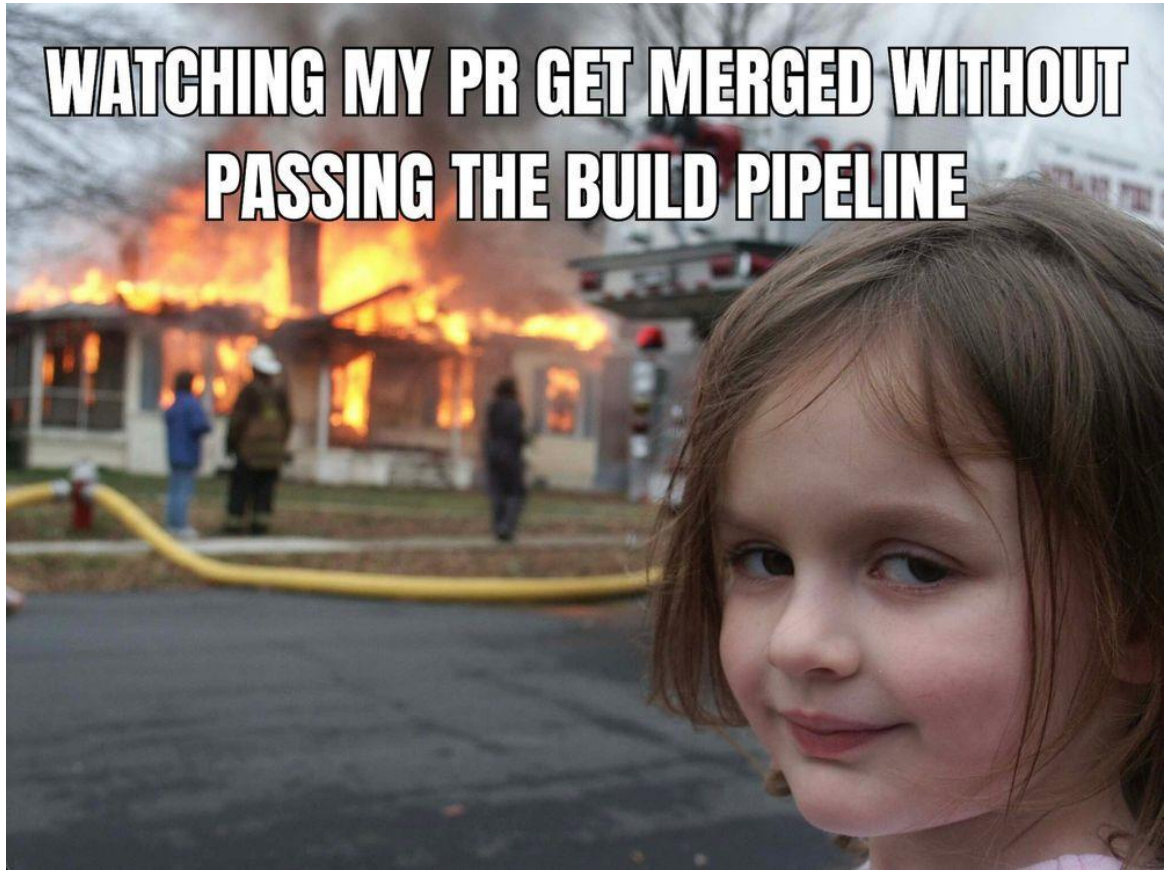


Proceso de desarrollo típico (en GitHub)

- Veamos un ejemplo
  - Una Pull Request puede validarse por otros miembros del equipo o, si el desarrollador tiene permiso, aprobarla directamente
  - Tras aprobar la Pull Request el código se mezcla en la rama destino

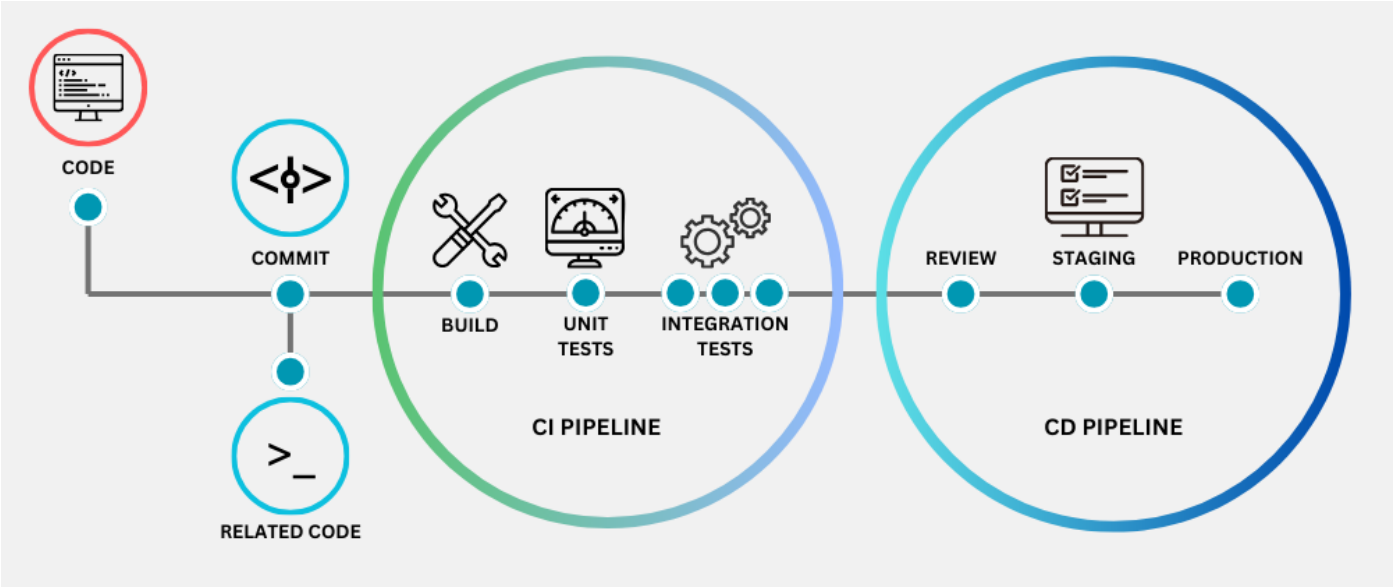
Proceso de desarrollo típico (en GitHub)

- Veamos un ejemplo
  - El código de un desarrollador con privilegios suficientes puede acabar en producción, pero…



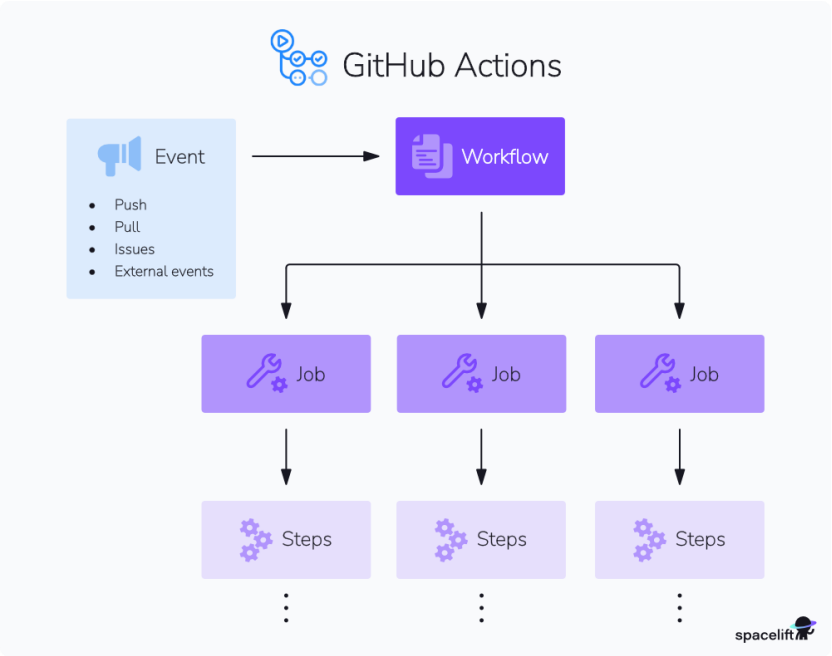
Proceso de desarrollo típico (en GitHub)

- Veamos un ejemplo
  - El código ha acabado en producción, pero:
    - ¿Quién asegura que el todo el proyecto, después de los cambios, funciona?
    - ¿Cómo aseguro que se mantengan los criterios de calidad establecidos en el proyecto?
    - Ese código que está en la rama de producción ¿puedo desplegarlo de manera automatizada?
  - Respuesta CI/CD (Integración y despliegue continuos)



Github Actions

- Plataforma de CI/CD
- Automatiza la construcción de un proyecto, verificación y despliegue
- Basado en la idea *workflow* asociado a *eventos*
  - Construir y probar después cada cambio en un pull request
  - Desplegar un pull request que se mezcla con master
  - En general, realizar acciones como respuesta a eventos en un repositorio



Ejemplo

.github/workflows/check-junit.yaml

```yaml
name: tests
on: push
jobs:
  run_tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout the repository
        uses: actions/checkout@v3
      - name: Set up JDK 21
        uses: actions/setup-java@v3
        with:
          java-version: 21
      - name: Run tests with Maven
        run: mvn -B test
```

Nombre del trabajo

Contenedor docker

Descargar repositorio en el contenedor

Añadir JVM al contenedor

Compilar y ejecutar las pruebas

Cualquier “step” con código de retorno != 0 aborta la ejecución

Ejemplo

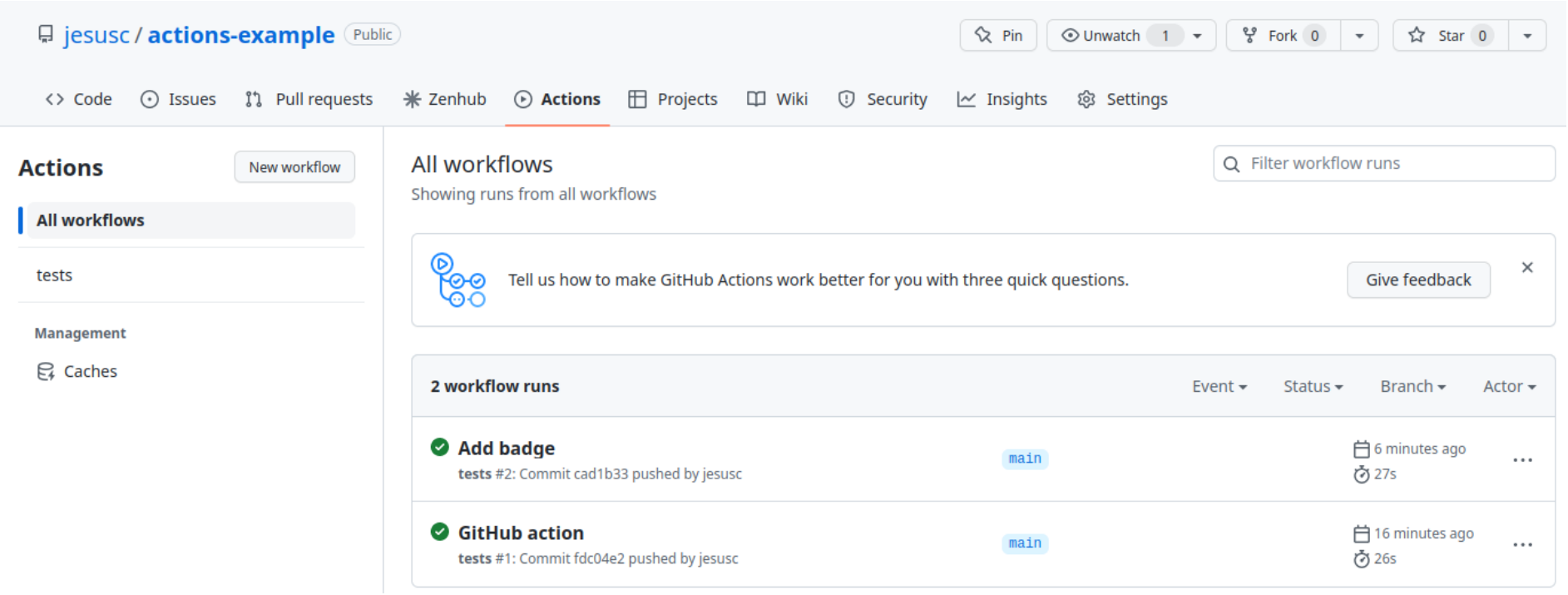
- Más cosas que se pueden hacen con Maven
  - https://docs.github.com/en/actions/automating-builds-and-tests/building-and-testing-java-with-maven

```
steps:
  - uses: actions/checkout@v3
  - uses: actions/setup-java@v3
    with:
      java-version: ‘11'
  - run: mvn --batch-mode --update-snapshots verify
  - run: mkdir staging && cp target/*.jar staging
  - uses: actions/upload-artifact@v3
    with:
      name: Package
      path: staging
```

Copiar y publicar artefactos

Ejemplo

- Comprobar los resultados de una acción



- Añadir un badge
  - `![example workflow](https://github.com/jesusc/actions-example/actions/workflows/check-junit.yaml/badge.svg)`

GitHub Actions

- ¿De dónde vienen las acciones?
  - Acciones built-in (ej., checkout)
  - Marketplace
    - https://github.com/marketplace?type=actions
    - Ejemplos:
      - https://github.com/marketplace/actions/close-stale-issues
      - https://github.com/marketplace/actions/prettier-java-action
  - Acciones propias definidas en un repositorio

PARTE II
Análisis estático y calidad

con SonarQube

SAST en desarrollos software – Motivación

Con **análisis estático de código** podemos prevenir:

- Problemas de programación que pueden dar lugar a errores.
- Vulnerabilidades en nuestro código.
- Malas prácticas de programación.
- Cobertura de los test.

- Diferentes herramientas:
  - SonarQube
  - Gitlab SAST
  - Github CodeQL
  - …

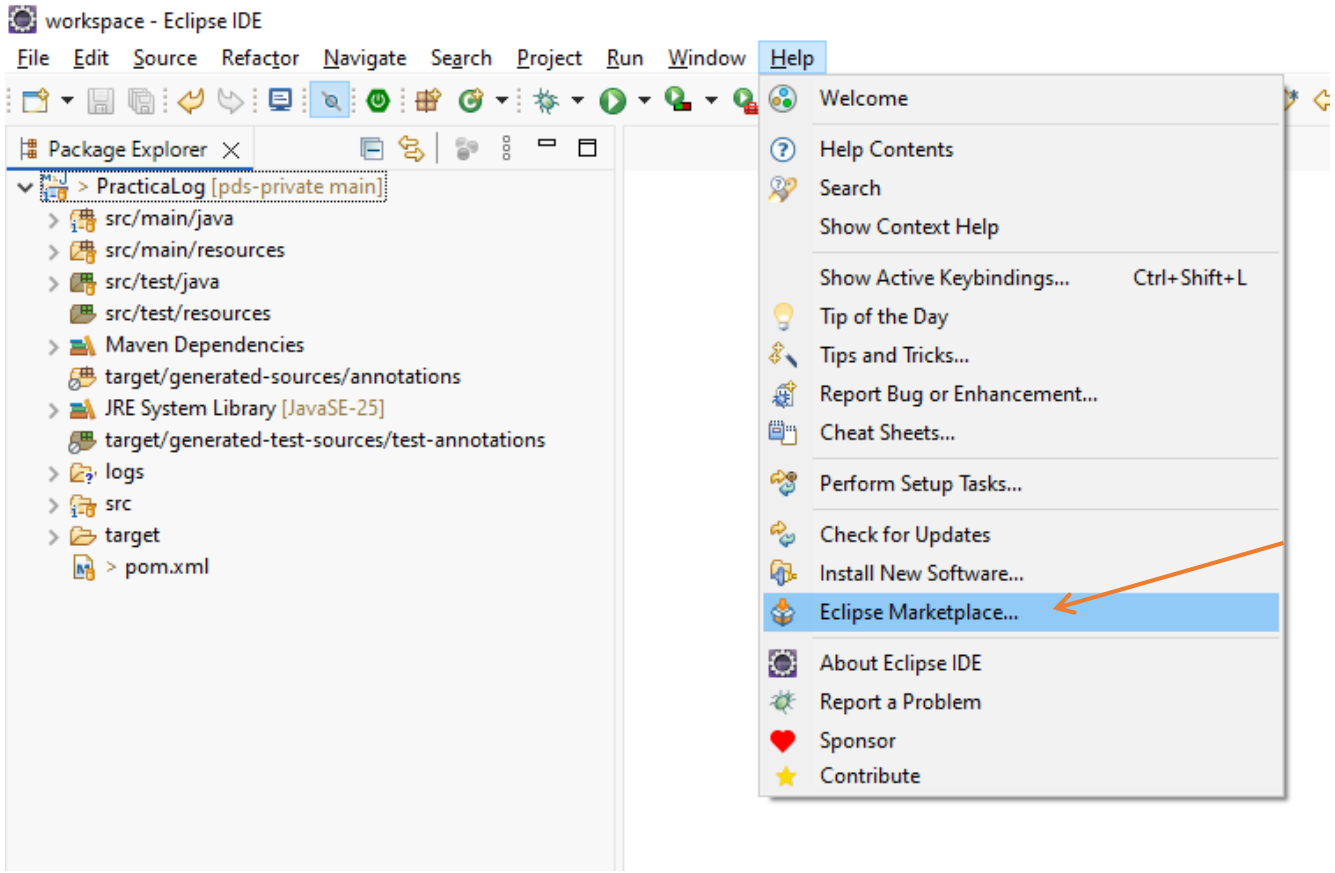
SAST en desarrollos software – SonarQube

En PDS vamos a usar **SonarQube** porque:

- Permite análisis en local ya que se integra con:
  - Eclipse
  - Visual Studio Code
  - IntelliJ
- Disponemos de un servidor SonarQube para la asignatura
- Puede integrarse con Maven en cualquier ciclo CI/CD

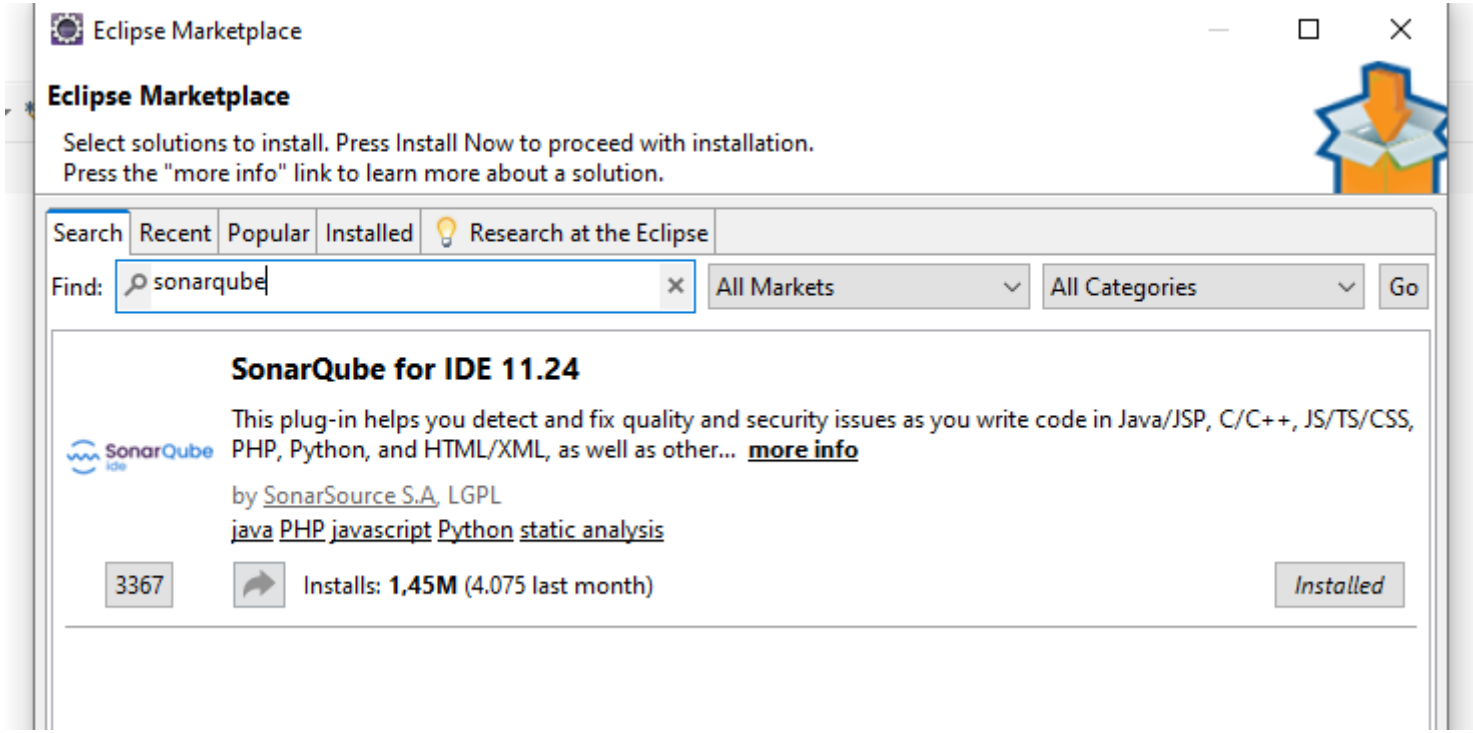
SAST – SonarQube – Configuración

- Instalación en eclipse:
  Accedemos al Marketplace



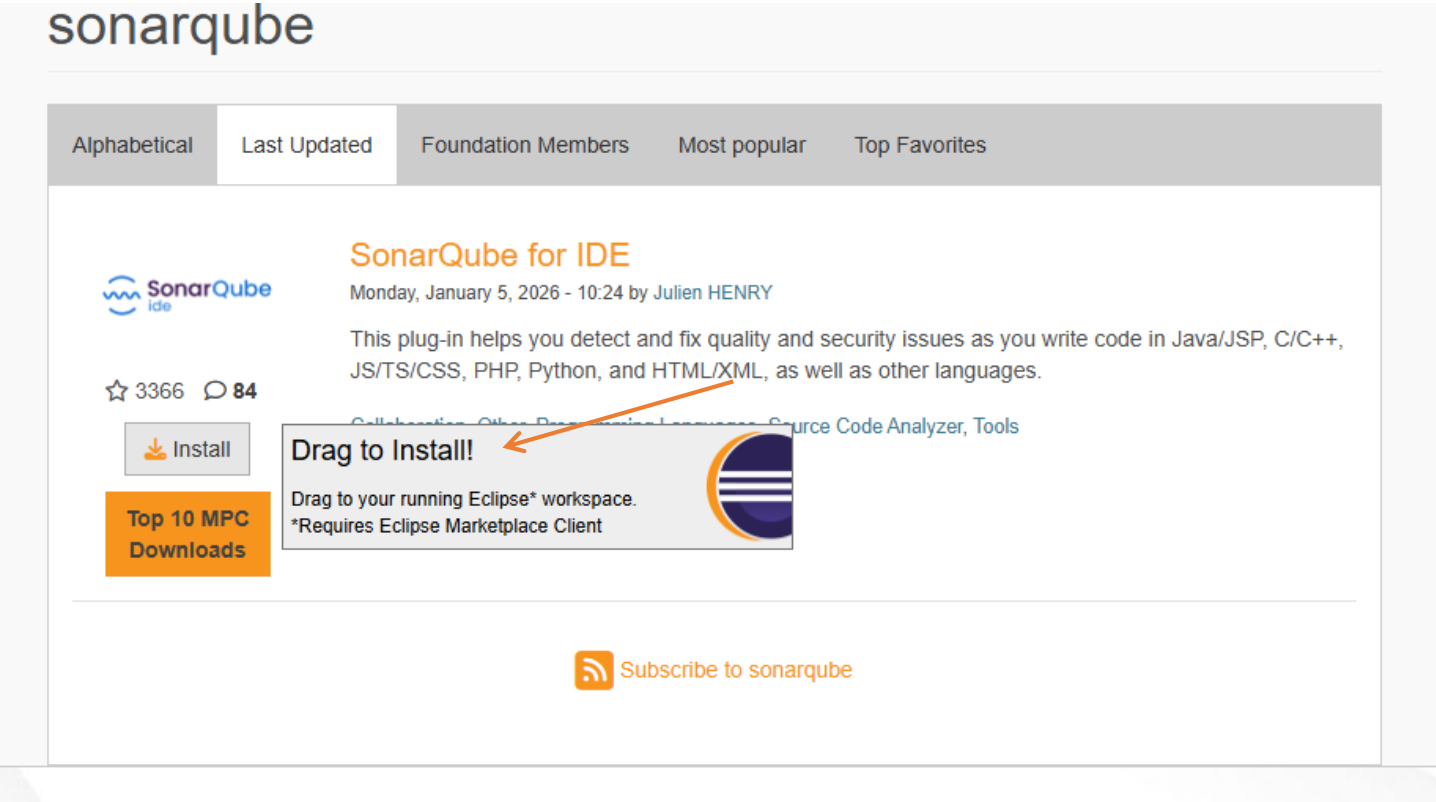
SAST – SonarQube – Configuración

- Instalación en eclipse:
  Buscamos "sonarqube" e instalamos el plugin.



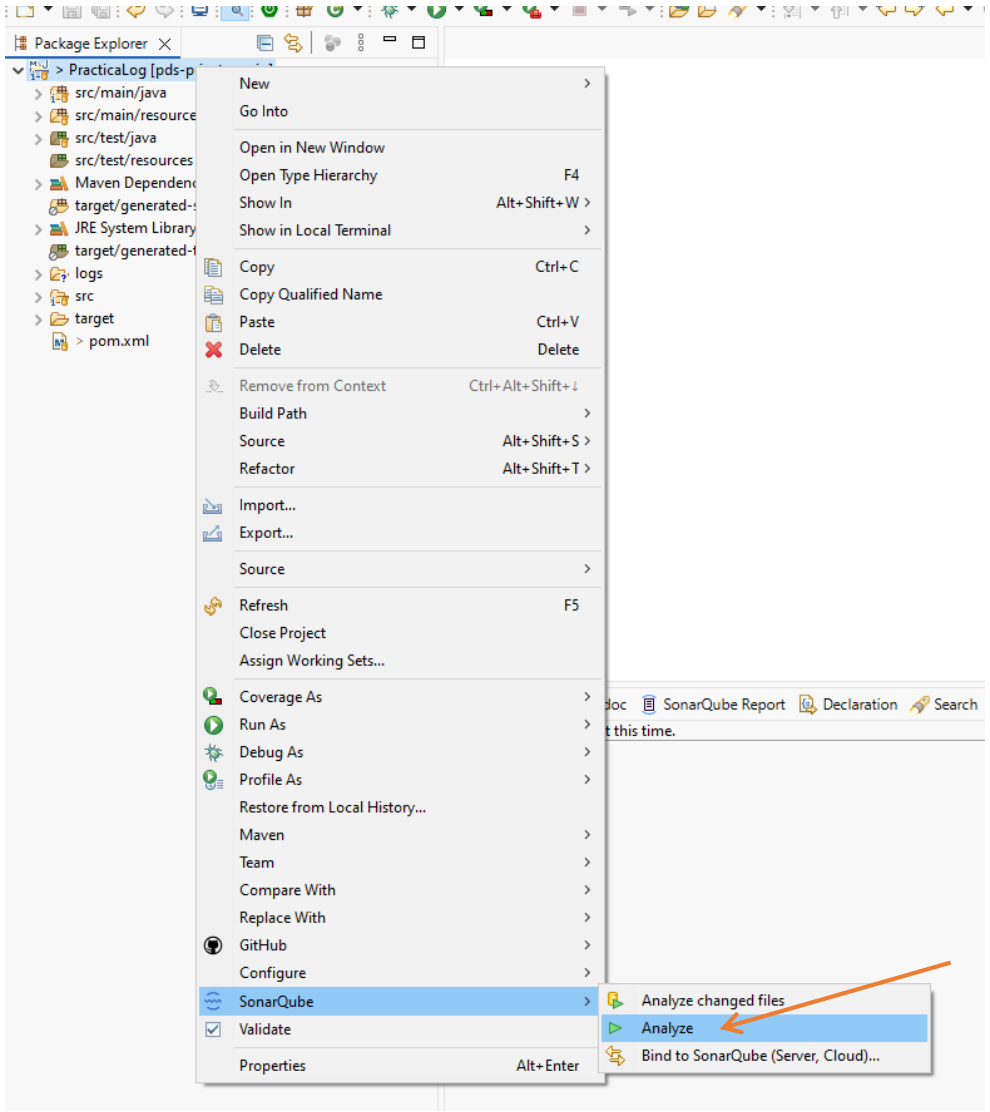
SAST – SonarQube – Configuración

- Instalación en eclipse:
  Instalación alternativa por si no aparece en el market
  Accedemos a: https://marketplace.eclipse.org/free-tagging/sonarqube



SAST – SonarQube – Uso

- Una vez instalado podremos escanear los proyectos en el menú contextual.
- Deberemos hacerlo sobre la raíz del proyecto para que lo escanee todo.
- Una vez acabe mostrará los resultados.



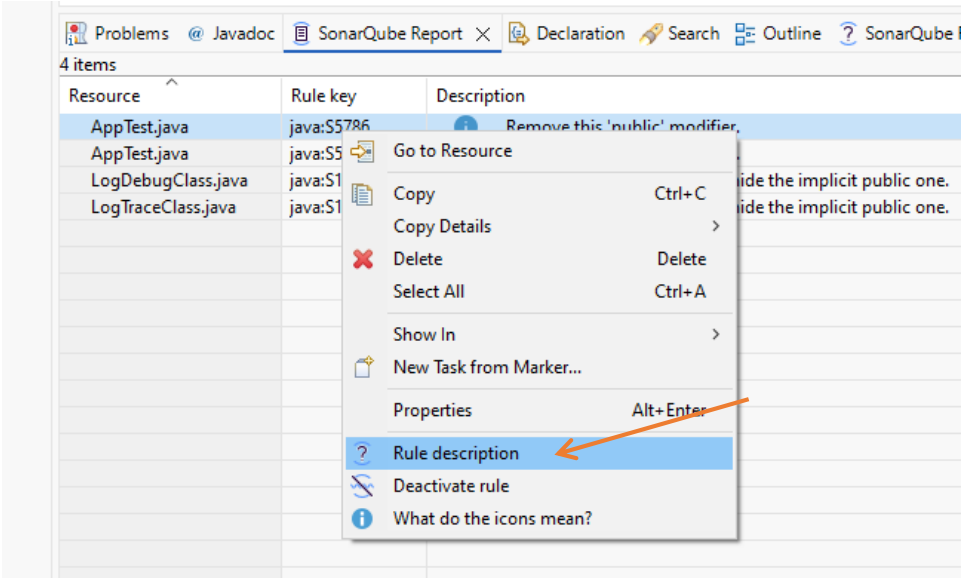
SAST – SonarQube – Uso

Nos listará los resultados y podremos consultar la regla que aplica.

Problems | Javadoc | SonarQube Report | Declaration | Search | Outline | SonarQube Rule Description | Console | SonarQube On-The-Fly

4 items

Resource	Rule key	Description	Date
AppTest.java	java:S5786	Remove this 'public' modifier.	23 hours ago
AppTest.java	java:S5786	Remove this 'public' modifier.	23 hours ago
LogDebugClass.java	java:S1118	Add a private constructor to hide the implicit public one.	1 day ago
LogTraceClass.java	java:S1118	Add a private constructor to hide the implicit public one.	1 day ago



SAST – SonarQube – Uso

Problems | Javadoc | SonarQube Report | Declaration | Search | Outline | SonarQube Rule Description | Console | SonarQube On-The-Fly

8 items

Resource	Rule key	Description	Date
App.java	java:S2259	A "NullPointerException" could be thrown; "pepe" is nullable here. [+2 locations]	few seconds ago
App.java	java:S2129	Remove this "String" constructor	few seconds ago
App.java	java:S1481	Remove this unused "b" local variable.	few seconds ago
App.java	java:S1854	Remove this useless assignment to local variable "b".	few seconds ago
AppTest.java	java:S5786	Remove this 'public' modifier.	23 hours ago
AppTest.java	java:S5786	Remove this 'public' modifier.	23 hours ago
LogDebugClass.java	java:S1118	Add a private constructor to hide the implicit public one.	1 day ago
LogTraceClass.java	java:S1118	Add a private constructor to hide the implicit public one.	1 day ago

SAST – SonarQube – Uso

Podemos conocer por qué salta la regla y cómo corregirlo.

Why is this an issue? | How can I fix it? | More Info

A reference to `null` should never be dereferenced/accessed. Doing so will cause a `NullPointerException` to be thrown. At best, such an exception will cause abrupt program termination. At worst, it could expose debugging information that would be useful to an attacker, or it could allow an attacker to bypass security measures.

Note that when they are present, this rule takes advantage of nullability annotations, like `@CheckForNull` or `@Nonnull`, defined in JSR-305 to understand which values can be null or not. `@Nonnull` will be ignored if used on the parameter of the `equals` method, which by contract should always work with null.

Why is this an issue? | How can I fix it? | More Info

Noncompliant code example

The variable `myObject` is equal to `null`, meaning it has no value:

```
public void method() {
  Object myObject = null;
  System.out.println(myObject.toString()); // Noncompliant: myObject is null
}
```

The parameter `input` might be `null` as suggested by the `if` condition:

```
public void method(Object input)
{
  if (input == null)
  {
    // ...
  }
  System.out.println(input.toString()); // Noncompliant
}
```

SAST – SonarQube – Uso

SonarQube también subrayará en azul, sobre nuestro código, los problemas potenciales que detecte

```
import umu.pds.debug.LogDebugClass;

public class App {

    public static void main(String[] args) {

        LogTraceClass.imprimeTrace();
        LogDebugClass.imprimeDebug();
        Object pepe = null;
        int a =0;
        int b =3;
        if(a>4) {
            pepe = new String();
        }
        pepe.toString();
    }
}
```

A "NullPointerException" could be thrown; "pepe" is nullable here.

3 quick fixes available:

- Show issue data flows
- Open description of rule java:S2259
- Deactivate rule java:S2259

Press 'F2' for focus

SAST – SonarQube – Integración

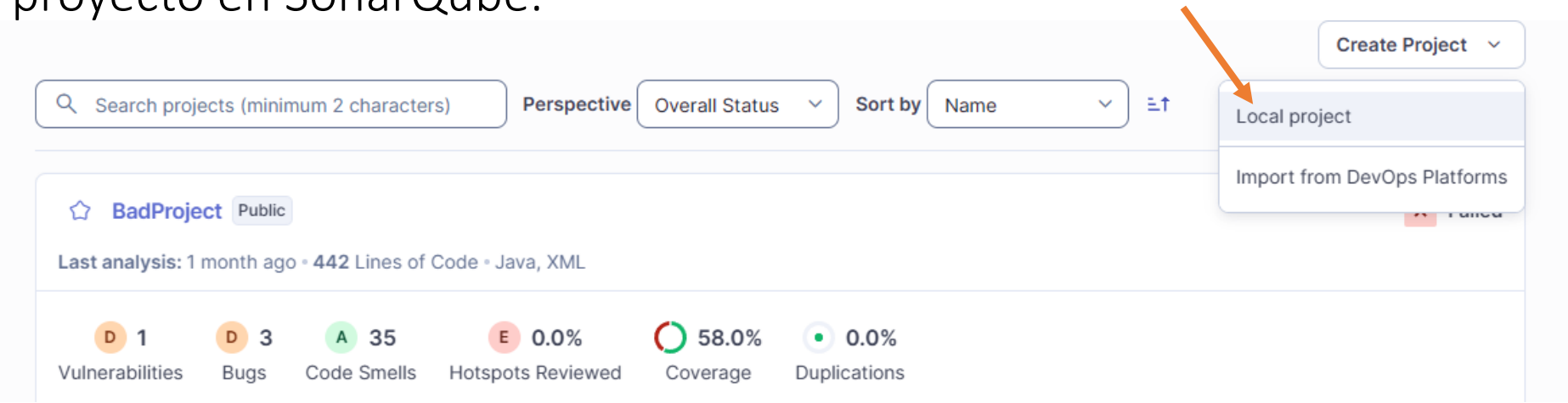
- Aunque el plugin de SonarQube es muy útil, en entornos de trabajo reales se limita a los entornos locales.
- Es necesario integrarlo en un servidor donde se centralice todos los desarrollos de la empresa o grupo de trabajo.
- Usado las herramientas CI/CD de las que se dispongan, podremos integrar el análisis de código dentro del ciclo de desarrollo de cualquier equipo.

SAST – SonarQube – Integración

- Vamos a analizar nuestro proyecto de dos maneras diferentes.
  - Enviándolo desde local al servidor SonarQube.
  - Añadiéndolo como una Github Action (CI/CD).
- Debemos subir nuestro proyecto a nuestro repositorio Github para poder hacer el segundo paso.

SAST – SonarQube – Integración

- El primer paso es configurar nuestro proyecto para que se analice en https://sast.inf.um.es
- Una vez hecho login con nuestra cuenta de la Universidad de Murcia creamos nuestro proyecto en SonarQube.

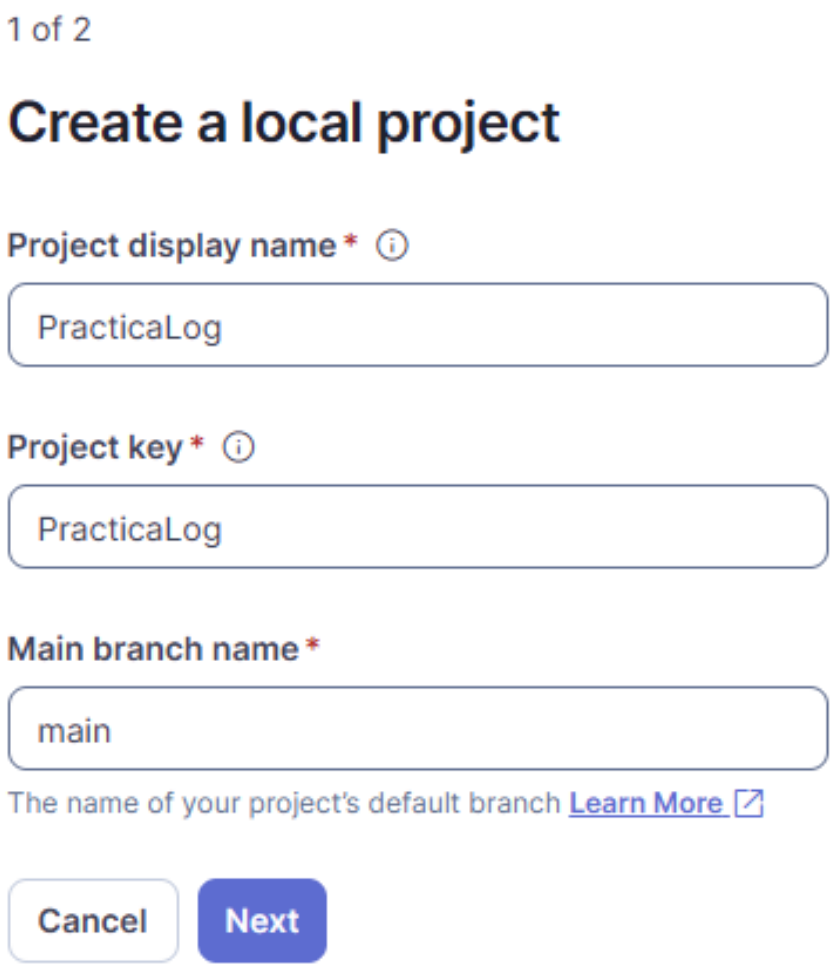


- Lo creamos como local porque no vamos a hacer un proceso de importación desde github.

SAST – SonarQube – Integración

Dentro del menú que nos aparece le tendremos que dar un nombre.

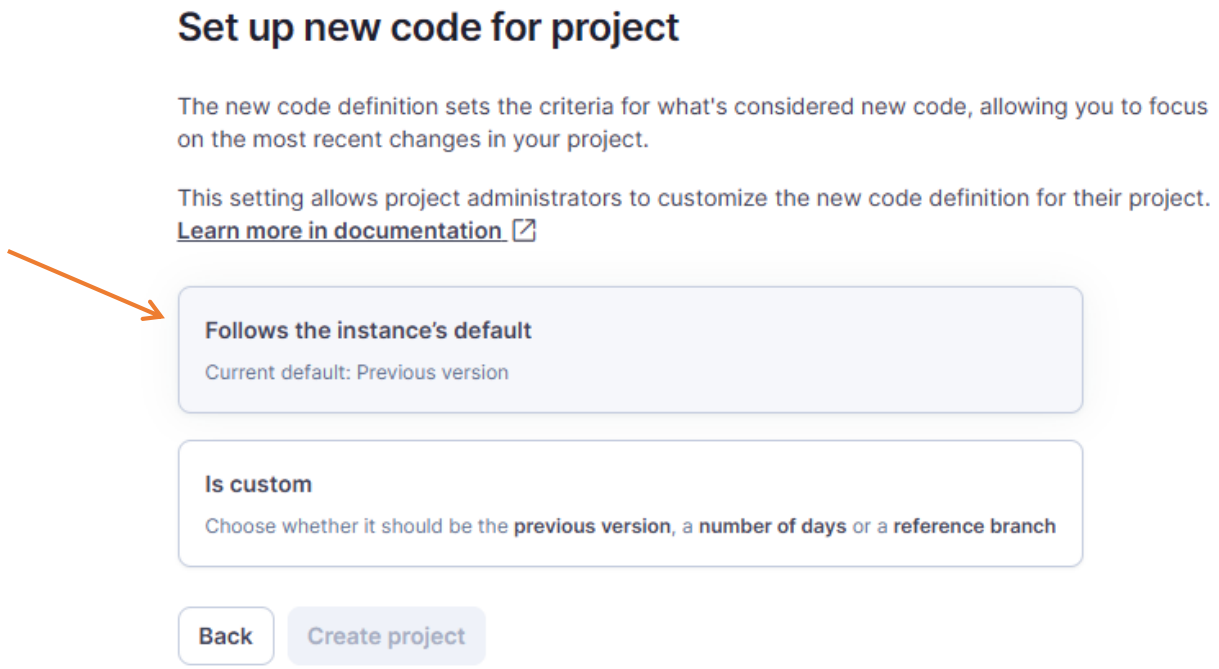
- Deberemos tener nombres de proyectos diferentes para que no colisionen
- Dejamos la rama principal con el valor por defecto "**`main`**"



SAST – SonarQube – Integración

Una vez hemos creado el proyecto, deberemos establecer el criterio para decidir qué es código nuevo

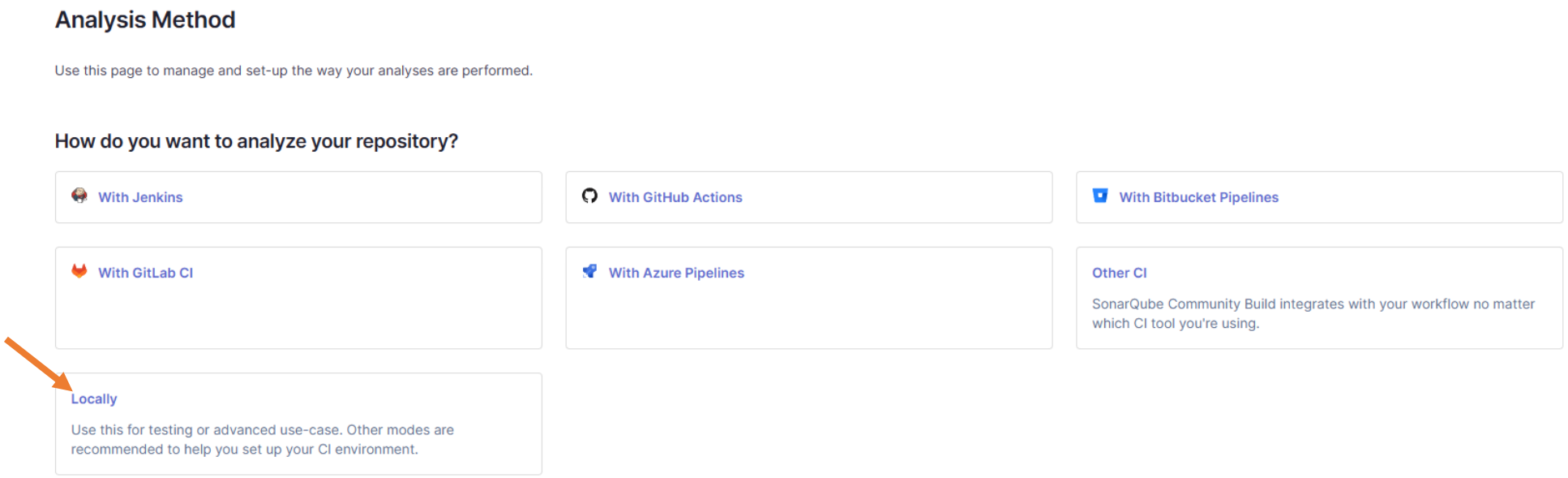
- Dejaremos por defecto el valor que tiene SonarQube



SAST – SonarQube – Integración

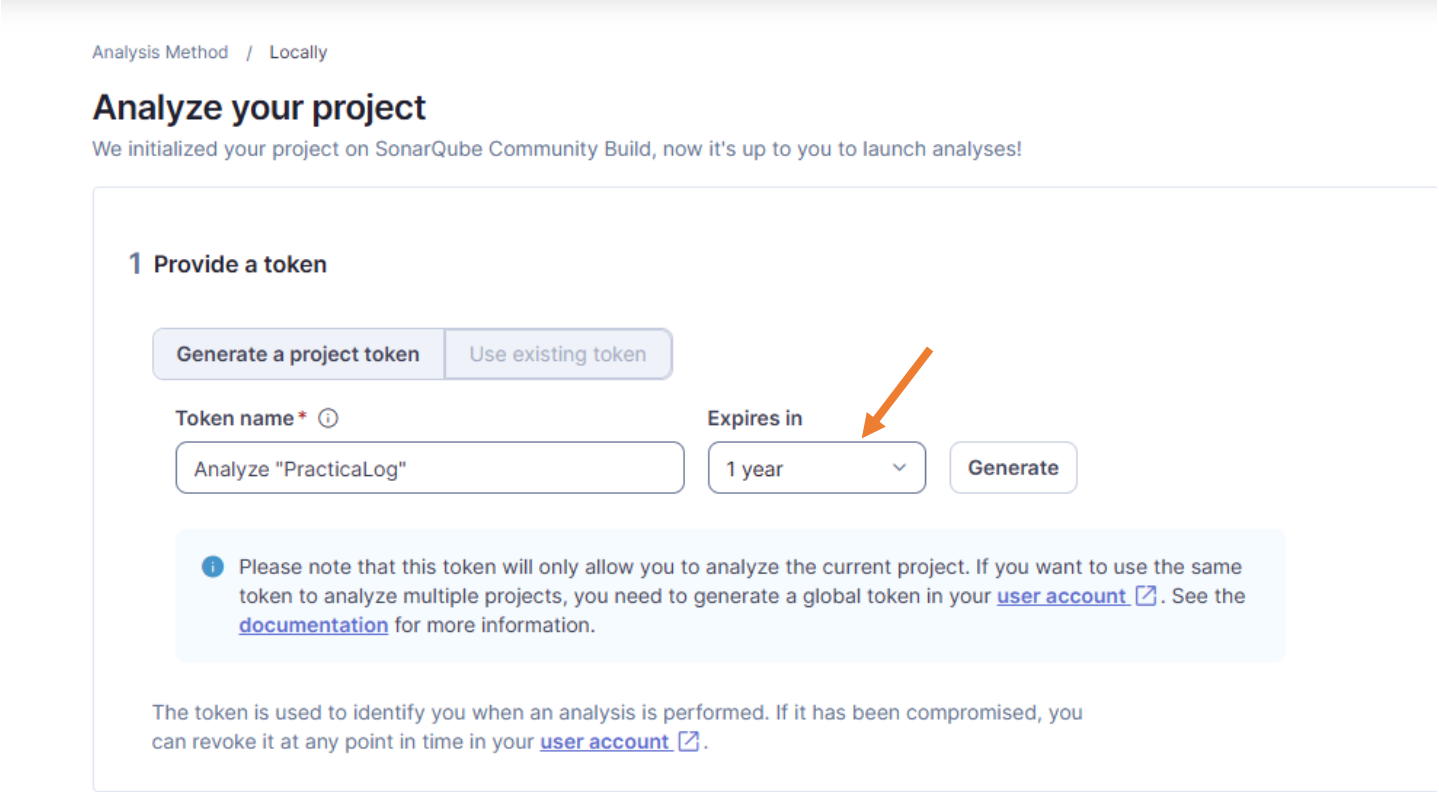
Con el proyecto configurado nos preguntará si queremos usar algún CI/CD existente para analizar el código.

- Nosotros marcaremos **local** ya que lo haremos vía Maven.



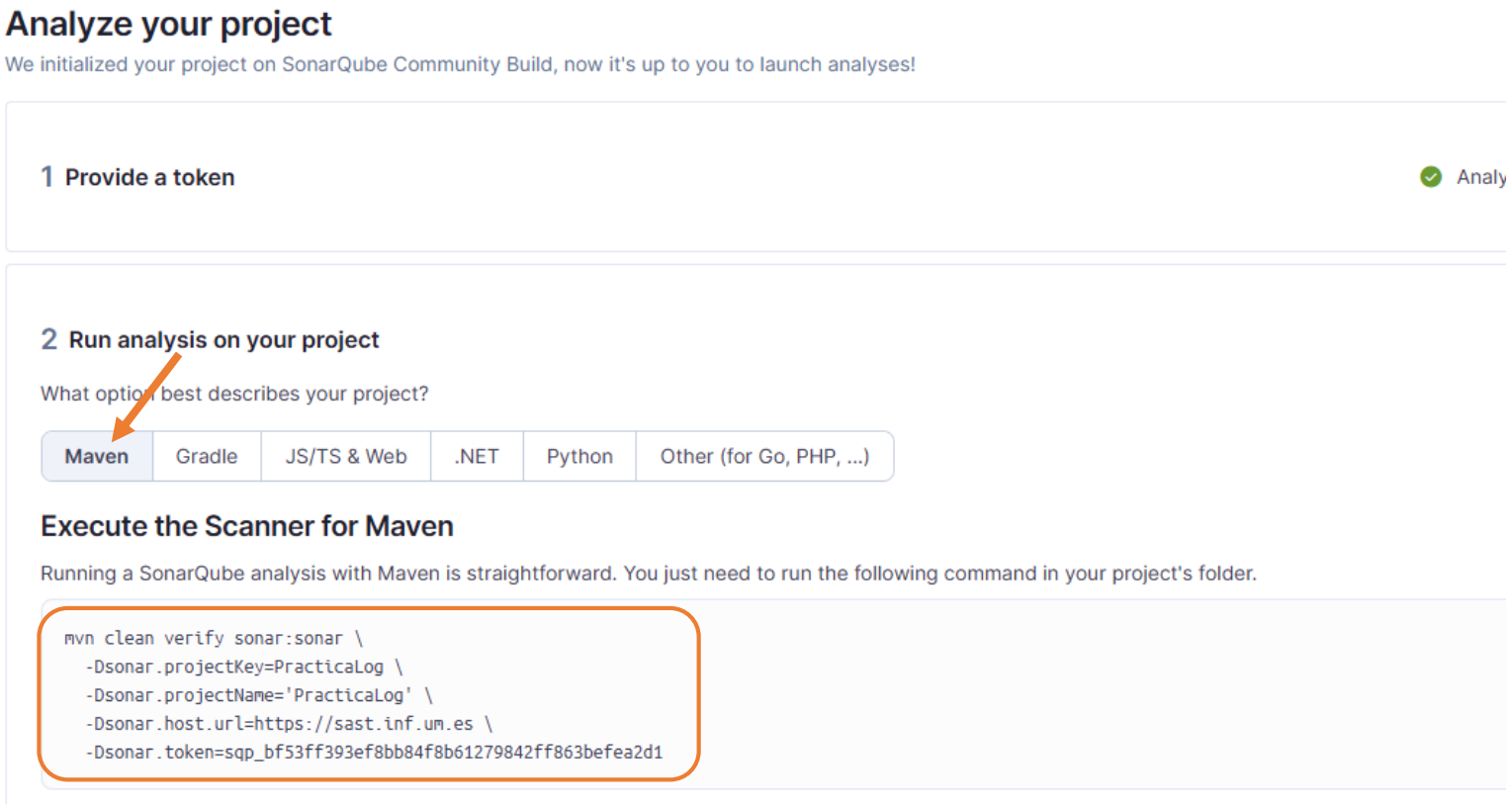
SAST – SonarQube – Integración

- El siguiente paso será generar un token para que podamos invocar a SonarQube para enviarle el código



SAST – SonarQube – Integración

- Por último, nos pregunta cómo queremos generar el análisis. Indicamos que con Maven



SAST – SonarQube – Integración

Lanzando el comando indicado desde local debería funcionarnos, pero en las últimas versiones hay que hacer un ajuste.

NOTA:

Si obtenemos el error:

```
[ERROR] No plugin found for prefix 'sonar' in the current project and in the plugin groups
[org.apache.maven.plugins, org.codehaus.mojo]
```

Debemos añadir el grupo de plugins a sonar. Para ello buscamos el fichero `settings.xml` de la instalación de Maven y añadimos dentro de `<settings>`

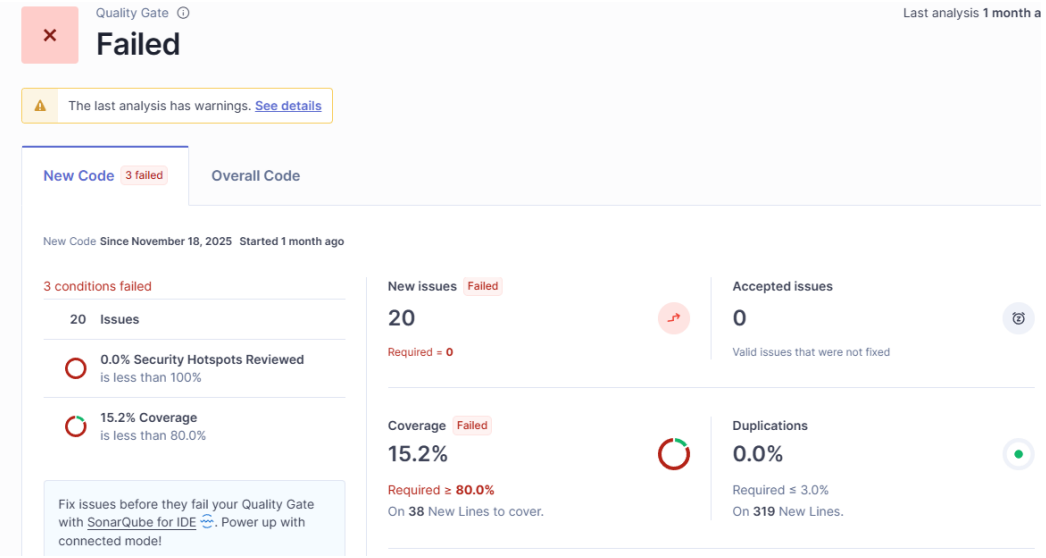
```
<pluginGroups>
    <pluginGroup>org.sonarsource.scanner.maven</pluginGroup>
</pluginGroups>
```

SAST – SonarQube – Integración

- Si continuamos teniendo problemas, podemos ejecutarlo indicando el plugin exacto.

```
mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:3.11.0.3922:sonar
-Dsonar.projectKey=[CLAVE_PROYECTO] -Dsonar.projectName=[NOMBRE_PROYECTO]
-Dsonar.host.url=https://sast.inf.um.es -Dsonar.token=[PONER_TOKEN]
```

- Una vez analizado, en SonarQube tendremos acceso a toda la información de ese código



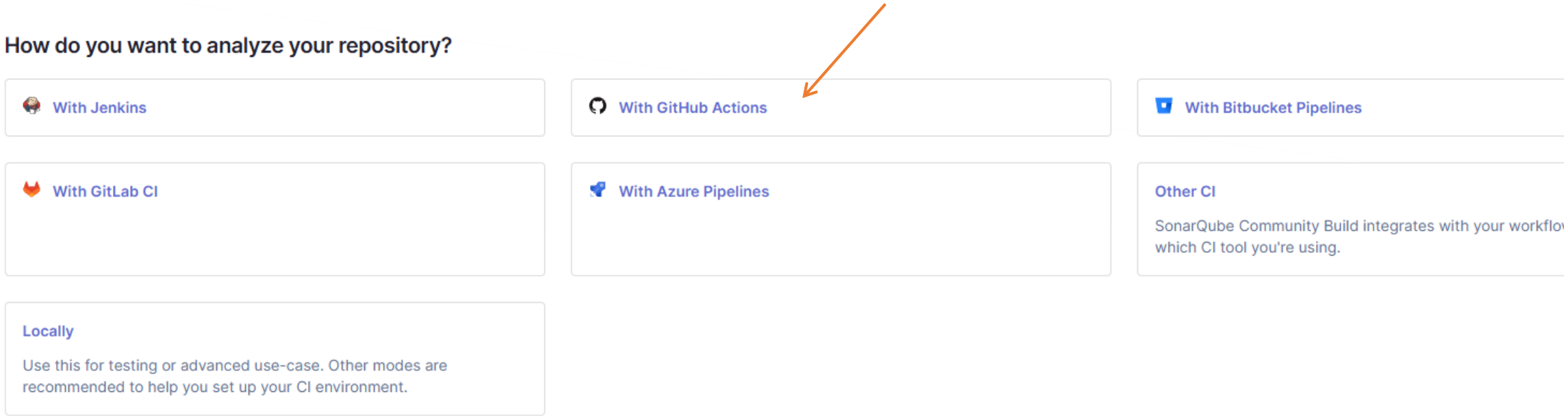
SAST – SonarQube – Integración

Una vez que sabemos enviar nuestro código al servidor SonarQube podemos integrarlo en Github Actions.

- SonarQube por defecto nos da una guía de los pasos a seguir.
- Si ya los conocemos podemos aplicarlo a nuestro proyecto existente, en caso contrario crearemos un nuevo proyecto para acceder a las indicaciones.

SAST – SonarQube – Integración

En la configuración del análisis, donde indicamos "Locally" ahora elegiremos "With GitHub Actions"



SAST – SonarQube – Integración

1. El primer paso es crear la configuración en Github

1 Create GitHub Secrets

In your GitHub repository, go to **Settings > Secrets and variables > Actions** and create the following new secrets:

1. Click on **New repository secret**.
2. In the **Name** field, enter `SONAR_TOKEN`
3. In the **Value** field, enter an existing token, or a newly generated one: **Generate a token**
4. Click on **Add secret**.

---

1. Click on **New repository secret**.
2. In the **Name** field, enter `SONAR_HOST_URL`
3. In the **Value** field, enter `https://sast.inf.um.es`
4. Click on **Add secret**.

SAST – SonarQube – Integración

1. El segundo paso es indicar cómo lo vamos a lanzar y SonarQube nos dará directamente el código de la **GitHub Action**.

1 What option best describes your project?

Maven	Gradle	JS/TS & Web	.NET	Python	Other (for Go, PHP, ...)

2 Create or update your `.github/workflows/build.yml` YAML file with the following content:

```yaml
name: Build

on:
  push:
    branches:
      - main


jobs:
  build:
    name: Build and analyze
    runs-on: ubuntu-latest
```

PARTE III
Reglas arquitecturales

con ArchUnit

ArchUnit

Chequeos de arquitectura

ArchUnit

- ¿Qué es ArchUnit?
  - Librería open-source para verificar la arquitectura de aplicaciones Java
  - Permite comprobar la arquitectura de nuestro proyecto lanzándose como un test unitario
  - Características:
    - Integrado con Maven
    - Inspecciona el código binario de las clases compiladas
    - Contiene un amplio abanico de reglas predefinidas
    - Permite crear nuestras propias reglas
    - Permite generar nuestros propios mensajes de error

```
<dependency>
   <groupId>com.tngtech.archunit</groupId>
   <artifactId>archunit-junit5</artifactId>
   <version>1.4.1</version>
   <scope>test</scope>
</dependency>
```

ArchUnit

- Detectar antes de tener que corregir
- ArchUnit nos permite crear reglas para
  - Mantener una correcta separación de capas
  - Evitar un alto acoplamiento entre módulos de nuestra aplicación
  - Evitar acumular código autogenerado por el IDE
  - Conseguir un nombrado coherente de nuestras clases

ArchUnit

- Proceso

ArchUnit

- ¿Cómo se compone una regla ArchUnit?

SUJETO	VERBO	RAZÓN

`classes()`
`.that()`
`.resideInAPackage`
`("..service..")`
	`.should()`
	`.beAnnotatedWith`
	`(@Service.class)`
		`.because("Los servicios`
		`deben estar anotados`
		`con @Service")`

¿Sobre qué clases aplica la regla?	¿Qué condición deben cumplir?	Mensaje en el error

ArchUnit

- Creación de mi primer test
  - Cuando creo un test ArchUnit es recomendable usar la anotación @AnalyzeClasses para limitar los paquetes que quiero que analice

```
@AnalyzeClasses(packages =“pds.umulingo")
public class UmuLingoArquitecturaTests { .. }
```

  - Con este código se restringe el análisis a las clases que están dentro del paquete `pds.umulingo` o hijos
  - Cada test debe tener la anotación @ArchTest
    - Puedo lanzar los test también haciendo uso de @Test, pero en este caso arrancará SpringBoot y no es necesario para hacer un análisis estático de las clases
    - El estándar de nombrado es poner las palabras separándolas por guiones bajos
    - Definiremos una variable estática y a la derecha de la asignación la ejecución de las reglas que dará como salida una ArchRule

```
@ArchTest
static final ArchRule ninguna_interfaz_acaba_en_impl = …
```

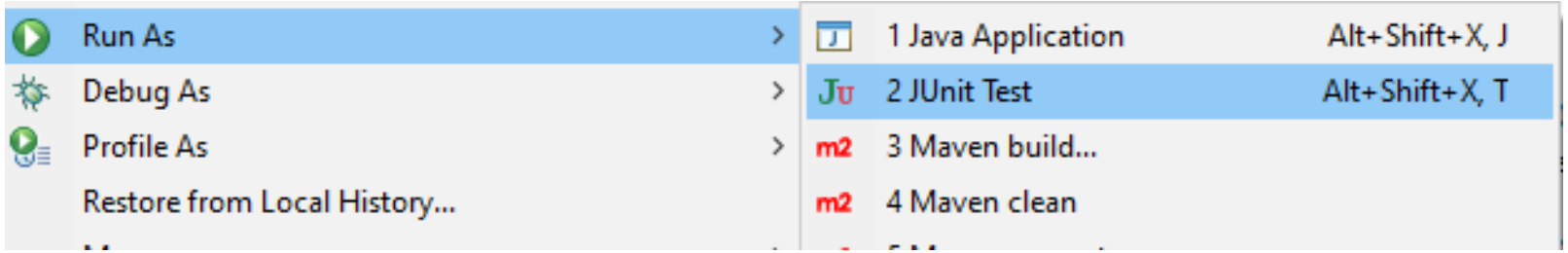
ArchUnit

- Creación de mi primer test
  - Una vez que defino mi primera regla tengo que seleccionar los elementos con los que quiero operar
  - ArchUnit por defecto provee métodos para:
    - Regla que deben cumplir todas las clases classes()
    - Regla que no debe cumplir ninguna clase noClasses()
    - Regla que deben cumplir todos los métodos methods()
    - Regla que no debe cumplir ningún método noMethods()
    - …
  - Todas las reglas iniciales podemos consultarla en la clase ArchRuleDefinition
  - Importante: En la documentación normalmente se importan esos métodos de manera estática, como otras reglas.
    - Normalmente en los ejemplos no se muestran los imports
    - Si nuestro código no compila al copiar los ejemplos debemos revisar que nuestros imports sean correctos

```
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.methods;
import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;
```

ArchUnit

- Creación de mi primer test
  - Para ejecutar los test es suficiente lanzarlos como un test unitario



Tema 5. ArchUnit

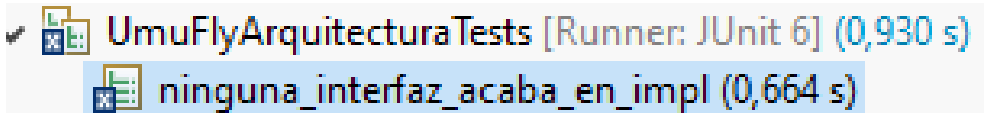
- Creación de mi primer test
  - Creemos una regla que haga que ninguna interfaz acabe en "Impl"
  - Iremos concatenando métodos que crearán la regla concreta
  - El nombre de los métodos es representativo, para poder entender la lógica de composición de las reglas
- Ejercicio:
  - Completad el test
  - Crear una interfaz que NO cumpla la regla
  - Probad que el test salta correctamente

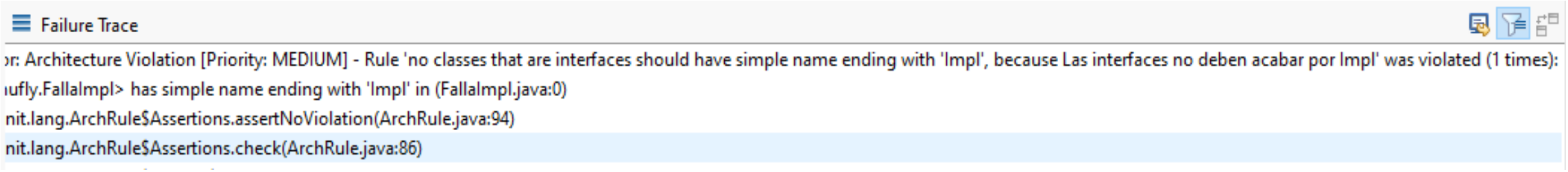
```
@AnalyzeClasses(packages = “pds.umulingo")
public class UmuLingoArquitecturaTests {

// Primer test ArchUnit
// Ninguna interfaz acaba en "Impl"
@ArchTest
static final ArchRule ninguna_interfaz_acaba_en_impl = ……;
}
```

Tema 5. ArchUnit

- Creación de mi primer test
- Ejercicio:
  - Completad el test
  - Crear una interfaz que NO cumpla la regla
  - Probad que el test salta correctamente
- El resultado esperado habiendo creado una interfaz acabada en Impl





Tema 5. ArchUnit

- Creación de mi primer test
  - Creemos una regla que haga que ninguna interfaz acabe en "Impl"
  - Iremos concatenando métodos que crearán la regla concreta
  - El nombre de los métodos es representativo, para poder entender la lógica de composición de las reglas

```
@AnalyzeClasses(packages = “pds.umulingo")
public class UmuLingoArquitecturaTests {

// Primer test ArchUnit
// Ninguna interfaz acaba en "Impl"
@ArchTest
static final ArchRule ninguna_interfaz_acaba_en_impl = noClasses().that().areInterfaces().should()
        .haveSimpleNameEndingWith("Impl").because("Las interfaces no deben acabar por Impl");
}
```

ArchUnit

- ArchUnit también nos permite poner reglas a nivel de estructura de código.
- Ofrece diversos tipos de arquitectura:
  - OnionArchitecture: Ideal para arquitecturas hexagonales puras, aunque poco flexible
  - LayeredArchitecture: Permite crear nuestra propia arquitectura por capas
- Vamos a implementar una LayeredArchitecture:
  - El método estático layeredArchitecture() es la base de la creación
  - Los métodos layer("nombre") y definedBy("ruta paquetes") definen cada capa
  - Los métodos whereLayer("nombre") y mayOnlyBeAccessedByLayers(lista de capas) indican, para cada capa cuáles pueden acceder a ella
  - El método mayNotBeAccessedByAnyLayer() indica que ninguna otra capa debe acceder a esa

ArchUnit

- Ejercicio:
  - Aplica LayeredArchitecture para definir una separación de capas, acorde a las arquitecturas hexagonales para tu proyecto de las prácticas
  - Crea alguna deficiencia en la arquitectura para que el test la detecte

ArchUnit

- Ejercicio:
  - Aplica LayeredArchitecture para definir una separación de capas, acorde a las arquitecturas hexagonales tu proyecto.
  - Crea alguna deficiencia en la arquitectura para que el test la detecte

```
static final ArchRule codigo_respeta_arquitectura_hexagonal = layeredArchitecture()
  // Cojo todas las dependencias del proyecto
  .consideringAllDependencies()
  // Defino la capa de dominio como las clases dentro del paquete domain
  .layer("Domain").definedBy("..domain..")
  // Defino la capa de aplicacion = paquete aplicacion
  .layer("Application").definedBy("..application..")
  // Adaptadores = clases dentro de adapters
  .layer("Adapters").definedBy("..adapters..")
  // Indico que "imports" de la
  // capa de dominio pueden estar en Application y Adapters
  .whereLayer("Domain").mayOnlyBeAccessedByLayers("Application", "Adapters")
  // Indico que "imports" de la capa de
  // adaptadores pueden estar en Application (pero no en dominio)
  .whereLayer("Application").mayOnlyBeAccessedByLayers("Adapters")
  // Ninguna capa debe tener import de adapters (lo gestiona Spring Boot)
  .whereLayer("Adapters").mayNotBeAccessedByAnyLayer();
```

ArchUnit

- ArchUnit también permite crear nuestras propias reglas haciendo uso de ArchCondition
- Las condiciones creadas se pueden añadir a las reglas de ArchUnit
- Ejercicio:
  - Comprobar que todo método dentro de una APIRest valide los parámetros de entrada

ArchUnit

- Ejercicio:
  - Comprobar que todo método dentro de una API REST valide los parámetros de entrada
- Primero obtenemos todos los métodos que estén en una API Rest

```
@ArchTest
static final ArchRule api_rest_debe_validar_datos_entrada =
methods().that().areDeclaredInClassesThat().areAnnotatedWith(RestController.class).and()
.arePublic()…
```

ArchUnit

- Ejercicio:
  - Comprobar que todo método dentro de una API REST valide los parámetros de entrada
- ArchUnit no tiene librería para comprobar si un método valida, vamos a crear nuestra propia ArchCondition

```
private static final ArchCondition<JavaMethod> METODO_REST_VALIDA_PARAMETROS =
new ArchCondition<>("Rest debe tener @Valid o @Validated en parámetros @RequestBody") {
        @Override
        public void check(JavaMethod metodo, ConditionEvents events) { … }
}
```

- En el constructor indico el nombre de la regla
- Debo implementar el método check

ArchUnit

- Ejercicio:
  - Comprobar que todo método dentro de una API REST valide los parámetros de entrada

```
public void check(JavaMethod metodo, ConditionEvents events) {
        boolean validaParametro = false;
        for (JavaParameter parametro : metodo.getParameters()) {
        //Compruebo si el metodo tiene RequestBody (JSON)
        if (parametro.isAnnotatedWith(RequestBody.class)) {
                validaParametro = parametro.isAnnotatedWith(Valid.class)
                        || parametro.isAnnotatedWith(Validated.class)
                        || metodo.getOwner().isAnnotatedWith(Validated.class);

                if (!validaParametro) {
                  String message = String.format("El método %s tiene un @RequestBody sin validación",
                  metodo.getFullName());
                  //Aniado un evento de violacion
                  events.add(SimpleConditionEvent.violated(metodo, message));
                }}
}}
```

ArchUnit

- Resumen:
  - Los test de arquitectura son muy útiles para mantener la coherencia del código
  - Debemos tener unas reglas claras y definidas para el proyecto
  - Detectan problemas en fases muy tempranas del desarrollo
  - Aseguran que el código no se degrada a nivel arquitectural

ArchUnit

- Ejercicio:
  - Crear un pequeño proyecto SpringBoot prototípico
  - Crear test que comprueben:
    - Las implementaciones de interfaces deben acabar en "Impl"
    - Los DTO acaban en DTO
    - Los RestController solo pueden estar en la capa de adaptadores
    - Ninguna clase debe tener más de 20 métodos públicos
  - Crear algunas clases de ejemplo que sean correctas arquitecturalmente y algunas clases que violen las reglas anteriores.